4.34

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    // 检查命令行参数是否正确
    if (argc != 2) {
        printf("用法: %s <文件名>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    // 打开文件
    FILE *fp = fopen(argv[1], "r");
    if (fp == NULL) {
        perror("无法打开文件");
        exit(EXIT_FAILURE);
    }
    // 读取并输出文件内容
    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
    // 关闭文件
    fclose(fp);
    return 0;
}
```

```
问题    输出    调试控制台    终端    端口

lenovo@LZH MINGW64 /d/C/Draft/demo1
$ ./cat test.txt
Test File
lenovo@LZH MINGW64 /d/C/Draft/demo1
$
```

4.35

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    // 检查命令行参数
    if (argc != 2) {
        printf("用法: %s <文件名>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    // 打开文件（二进制只读模式）
    FILE *fp = fopen(argv[1], "rb");
    if (fp == NULL) {
        perror("文件打开失败");
        exit(EXIT_FAILURE);
    }
    int ch;
    int count = 0; // 记录当前行已输出的字节数
    // 逐个读取字节并输出十六进制
    while ((ch = fgetc(fp)) != EOF) {
        printf("%02x ", ch); // 两位十六进制，不足补0
        count++;
        if (count % 40 == 0) { // 每40字节换行
            printf("\n");
        }
    }
    // 最后一行如果不足40字节，补充换行
    if (count % 40 != 0) {
        printf("\n");
    }
    // 关闭文件
    fclose(fp);
    return 0;
}
```

```
lenovo@LZH MINGW64 /d/C/Draft/demo1
$ ./Hex test.txt
54 65 73 74 20 46 69 6c 65

lenovo@LZH MINGW64 /d/C/Draft/demo1
$
```

4.36

loc1=10 ch=k, loc2=27

4.37

（1）k * sizeof(T)
（2）k * sizeof(T)
（3）

```c
// 定义变量
T *e = (T *)malloc(sizeof(T)); // 分配内存

// 定位到a[k]的位置
lseek(fd, k * sizeof(T), SEEK_SET);
// 读取内容到e指向的内存
read(fd, e, sizeof(T));
```

4.38

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>   // lseek、read、write、close
#include <fcntl.h>    // open

typedef struct subject {
    char sno[20];
    char name[20];
    float chinese;
```

```
    float math;
    float english;
} subject;
int main() {
    subject stu[5];
    int fd;
    printf("请输入 5 个学生信息: \n");
    for (int i = 0; i < 5; i++) {
        scanf("%s %s %f %f %f",
            stu[i].sno,
            stu[i].name,
            &stu[i].chinese,
            &stu[i].math,
            &stu[i].english);
    }
    fd = open("data", O_WRONLY | O_CREAT, 0666);
    write(fd, stu, sizeof(stu));
    close(fd);
    fd = open("data", O_RDONLY);
    subject temp;
    lseek(fd, 0 * sizeof(subject), SEEK_SET);
    read(fd, &temp, sizeof(subject));
    printf("第 1 条: %s %s %.1f %.1f %.1f\n",
        temp.sno, temp.name, temp.chinese, temp.math, temp.english);
    lseek(fd, 2 * sizeof(subject), SEEK_SET);
    read(fd, &temp, sizeof(subject));
    printf("第 3 条: %s %s %.1f %.1f %.1f\n",
        temp.sno, temp.name, temp.chinese, temp.math, temp.english);
    lseek(fd, 3 * sizeof(subject), SEEK_SET);
    read(fd, &temp, sizeof(subject));
    printf("第 4 条: %s %s %.1f %.1f %.1f\n",
        temp.sno, temp.name, temp.chinese, temp.math, temp.english);
    close(fd);
    return 0;
}
```

4.39

```
#include <fcntl.h>
#include <unistd.h>

void write_to_file(T var[N], int B) {
    int fd = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
    if (fd == -1) {
        // 错误处理
        return;
    }

    int total_bytes = N * sizeof(T);
    int written = 0;

    while (written < total_bytes) {
        int write_size = (total_bytes - written) > B ? B : (total_bytes - written);
        write(fd, var + written / sizeof(T), write_size);
        written += write_size;
    }

    close(fd);
}
```

4.40

open 函数执行过程：

1. 内核检查文件是否存在、权限是否合法。
2. 为进程分配一个文件描述符（fd）。
3. 内核创建并填充 `file` 结构体（包含文件偏移量、访问模式等），并将其与文件描述符关联。
4. 返回文件描述符。

close 函数执行过程：

1. 内核回收文件描述符对应的 `file` 结构体。
2. 如果该文件的引用计数变为 0，内核会关闭文件对应的 inode，并释放相关资源。

为何读写文件前要打开文件：

打开文件是为了让内核建立进程与文件的关联，分配文件描述符，记录文件的访问状态（如偏移量、权限）。没有打开文件，进程无法直接对文件进行读写操作，内核也无法管理文件的访问状态。

4.41

```
The oldfd1 file descriptor=3
The oldfd2 file descriptor=4
The newfd file descriptor=5
The newfd2 file descriptor=0
I have read from a.txt:abcdefgh
```

4.42

权限为：rw------x

4.43

文件内容是：789ab

4.44

fd1=3,fd2=4
fd3=3

4.45

fd2=5

4.46

c1=s
c2=s

4.47

fd2=4
fd3=0

4.48

2016 LINUX EXAM GOOD!GOOD!

4.49

(1) M=(M2-M1)/N

(2) T=[(M2-M1)- Δ 2- Δ 1]/N

4.50

```
#include <stdio.h>
#include <sys/time.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUF_SIZE 1024

long long time_diff(struct timeval *start, struct timeval *end) {
	return (end->tv_sec - start->tv_sec) * 1000000 + (end->tv_usec - start->tv_usec);
}

int main() {
	int fd;
	FILE *fp;
	char buf[BUF_SIZE];
	struct timeval start, end;
	long long read_time, fread_time;

	fd = open("test.txt", O_RDONLY);
	if (fd == -1) {
		perror("open failed");
		exit(1);
	}
	gettimeofday(&start, NULL);
	read(fd, buf, BUF_SIZE);
	gettimeofday(&end, NULL);
	read_time = time_diff(&start, &end);
	close(fd);

	fp = fopen("test.txt", "r");
	if (fp == NULL) {
		perror("fopen failed");
		exit(1);
```

```
    }
    gettimeofday(&start, NULL);
    fread(buf, 1, BUF_SIZE, fp);
    gettimeofday(&end, NULL);
    fread_time = time_diff(&start, &end);
    fclose(fp);

    printf("read()  执行时间: %lld  微秒\n", read_time);
    printf("fread()执行时间: %lld  微秒\n", fread_time);

    return 0;
}
```

```
read() 执行时间: 12 微秒
fread()执行时间: 8 微秒
```

解释：

read()：作为系统调用，执行时会触发用户态到内核态的上下文切换，单次调用的耗时通常在几微秒到几十微秒。

fread()：作为库函数，内部会维护缓冲区，单次调用的耗时可能比 read()略高（因为有库函数的封装逻辑），但在多次读取时，因为减少了系统调用次数，整体效率更高。